

Arduino Modbus Test

Purpose

This test establishes a framework for Modbus communications on the Arduino Opto which can be used for future integration. Holding registers and input registers are set up to demonstrate the ability to read telemetry data and to send commands. A full cycle of command and response is tested, enabling us to verify that the command was received properly, and a heartbeat is also established to verify that the communication channel remains open.

Description

In this test, the Arduino Opto acts as a Modbus client, while a Modbus server is running on the testing laptop. This server is set up using OpenModSim. The laptop will run at IP address 172.16.0.1 and the Arduino will run at IP address 172.16.0.2

OpenModSim will be set up to run with one input register. It will send a device ID with a value of 1234. This register is used to test telemetry reporting. The device ID will be at input register 0.

There will be four holding registers. The first will send a heartbeat counter from the Arduino to OpenModSim. This counter updates from 1 to 10, incrementing once per second and resetting from 10 to 1 when it finishes. The heartbeat is sent on holding register 0. This tests heartbeat functionality.

The command will be sent from OpenModSim to the Arduino on holding register 1. The Arduino will then reflect that command on holding register 2. The Arduino will then multiply the command by 2 and send that value out on holding register 3. This will establish that a command can be sent and verified by reflecting it, and that the Arduino is able to take an action based on that command and send a result back.

The test consists of first reading the device ID to confirm that telemetry is being received. Then the heartbeat is read at least twice to confirm that it is updating by verifying that it shows two different values at two different times. Next, a command value of 10 is sent from the laptop. The reflected value will then be read, expecting a value of 10. The doubled value will then be read, expecting a value of 20. Finally, a command value of 3 is sent from the laptop. The reflected value will be read, expecting a value of 3. The doubled value is read, expecting a value of 6.

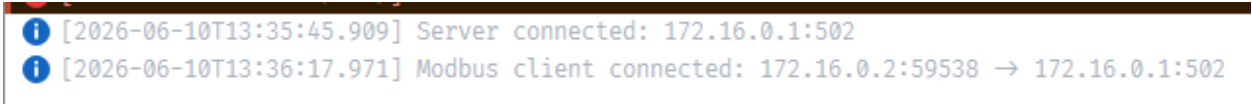
Procedure

	Step	Description	Result
--	------	-------------	--------

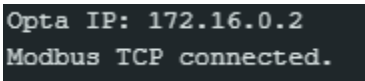
0	Initial Conditions	The Arduino and the laptop are connected to the same subnet using ethernet cables. The Modbus connection between the Arduino and the laptop has been established. Attach a screenshot from OpenModSim in the Result column to show that the connection is established.	See below
1	Read telemetry	Use OpenModSim to write 1234 to input register 0. Attach a screenshot from the Opta in the Result column to show that the expected value was received.	See below
2	Read heartbeat counter	Use OpenModSim to read holding register 0. The expected value is between 1 and 10. Attach a screenshot from OpenModSim in the Result column to show that the expected value was received.	See below
3	Confirm heartbeat counter is incrementing	Use OpenModSim to read holding register 0 again. The expected value is between 1 and 10, and it should be a different value than that read in step 2. Attach a screenshot from OpenModSim in the Result column to show that the expected value was received.	See below
4	Send command of 10	Use OpenModSim to send a value of 10 to holding register 1. Read holding registers 2 and 3. The expected value of holding register 2 is 10. The expected value of holding register 3 is 20. Attach a screenshot from OpenModSim in the Result column to show that the expected value was received.	See below
5	Send command of 3	Use OpenModSim to send a value of 3 to holding register 1. Read holding registers 2 and 3. The expected value of holding register 2 is 3. The expected value of holding register 3 is 6. Attach a screenshot from OpenModSim in the Result column to show that the expected value was received.	See below
6	Tear Down	Use OpenModSim to close the connection	See below

		between the laptop and the Arduino. Unplug the ethernet cables from the Arduino and the laptop. Attach a screenshot from ModbusPal in the Result column to show that the connection is closed.	
--	--	--	--

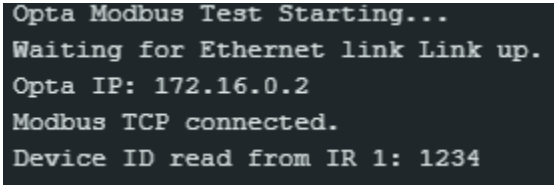
Step 0: Connection Verification from OpenModSim



Step 0: Connection Verification from Opta



Step 1: Read Telemetry



Step 2: Read heartbeat

255	Holding Registers	0	Int16		Direct		1	2026-06-12T10:50:59.857
-----	-------------------	---	-------	--	--------	--	---	-------------------------

Step 3: Confirm heartbeat updates

255	Holding Registers	0	Int16		Direct		8	2026-06-12T10:51:16.856
-----	-------------------	---	-------	--	--------	--	---	-------------------------

Step 4: Confirm command 10

255	Holding Registers	1	Int16		Direct		10	2026-06-11T12:47:19.001
255	Holding Registers	2	Int16		Direct		10	2026-06-11T12:47:36.475
255	Holding Registers	3	Int16		Direct		20	2026-06-11T12:47:36.485

Step 5: Confirm command 3

255	Holding Registers	1	Int16	Direct	3	2026-06-12T10:51:51.143
255	Holding Registers	2	Int16	Direct	3	2026-06-12T10:51:51.365
255	Holding Registers	3	Int16	Direct	6	2026-06-12T10:51:51.374

Step 6: Disconnect

[2026-06-12T10:52:17.785] Modbus client disconnected: 172.16.0.2:53540 → 172.16.0.1:502
[2026-06-12T10:52:17.785] Server disconnected: 172.16.0.1:502

Arduino MQTT Test

Purpose

This test establishes a framework for MQTT communications on the Arduino Opto which can be used for future integration. Subscriptions and publications are set up to demonstrate the ability to read telemetry data and to send commands. A full cycle of command and response is tested, enabling us to verify that the command was received properly, and a heartbeat is also established to verify that the communication channel remains open.

Description

In this test, the testing laptop runs the MQTT broker. This broker is set up using Mosquitto. The laptop will run at IP address 172.16.0.1 and the Arduino will run at IP address 172.16.0.2.

The connection will be verified using the opta/status topic. The Arduino will publish "online" to this topic when it connects. The Arduino will also establish a Last Will protocol so that the broker will publish "offline" to this topic if the Arduino disconnects unexpectedly.

Telemetry and heartbeat functionality are tested by having the Arduino publish to two topics. The first topic is a device id at opta/device-id, which will publish a device ID with a value of 1234 when it detects a connection. The second topic is at opta/heartbeat, and will send a heartbeat counter updating from 1 to 10, incrementing once per second and resetting from 10 to 1 when it finishes.

Commands are tested by having the Arduino subscribe to one topic and publish to two other topics. The command will be sent from the laptop to the Arduino on the topic opta/commands/input. The laptop publishes to this topic and the Arduino subscribes to it. The Arduino reflects that command back to the laptop by publishing on opta/commands/echo. The Arduino will then multiply the command by 2 and publish that value out on the topic opta/commands/result. This will establish that a command can be sent and verified by reflecting it, and that the Arduino is able to take an action based on that command and send a result back.

The test consists of first reading the device ID to confirm that telemetry is being received. Then the heartbeat is read at least twice to confirm that it is updating by verifying that it shows two different values at two different times. Next, a command value of 10 is sent from the laptop. The reflected value will then be read, expecting a value of 10. The doubled value will then be read, expecting a value of 20. Finally, a command value of 3 is sent from the laptop. The reflected value will be read, expecting a value of 3. The doubled value is read, expecting a value of 6.

Procedure

	Step	Description	Result
0	Initial Conditions	The Arduino and the laptop are connected to the same subnet using wifi. The MQTT configuration file has been written and saved at C:\mosquitto\test.conf.	
1	Start the Mosquitto MQTT server	Open a command line interface. Enter the command mosquitto -c C:\mosquitto\test.conf -v . Leave this command window open. It operates as the broker.	
2	Verify MQTT connection	Open a second command line interface. Enter the command mosquitto_sub -h 172.16.0.1 -t "opta/status" -v to subscribe to the status of the Opta and read the stored value. The command should return opta/status online . Attach a screenshot of the second command window to show that the expected response was received.	See below
3	Read telemetry	Enter the command mosquitto_sub -h 172.16.0.1 -t "opta/device-id" -v --retained-only to subscribe to the device id of the Opta and read the stored value. The command should return opta/device-id 1234 . Attach a screenshot of the second command window to show that the expected response was received.	See below
4	Read heartbeat counter	Enter the command mosquitto_sub -h 172.16.0.1 -t "opta/heartbeat" -v -C 1 to subscribe to the heartbeat of the Opta and read the stored value. The command should return opta/heartbeat # , where # is a number between 1 and 10. Attach a screenshot of the second command window to show that the expected response was received.	See below
5	Confirm heartbeat counter is incrementing	Wait a couple of seconds and then enter the command mosquitto_sub -h 172.16.0.1 -t "opta/heartbeat" -v -C 1 to subscribe to the heartbeat of the Opta and read the stored value.	See below

		The command should return opta/heartbeat #, where # is a number between 1 and 10. This number should be different than the number read in step 4. Attach a screenshot of the second command window to show that the expected response was received.	
6	Send command of 10	Open a third and fourth command line prompt to read the results of the commands. Send the following command in the third command window to subscribe to the echo response: mosquitto_sub -h 172.16.0.1 -t "opta/commands/echo" -v -C 1. Send the following command to the fourth command window to subscribe to the doubled result: mosquitto_sub -h 172.16.0.1 -t "opta/commands/result" -v -C 1. Send the following command to the second command window to publish the input to the Opta: mosquitto_pub -h 172.16.0.1 -t "opta/commands/input" -m "10". Window 3 should show a result of opta/commands/echo 10. Window 4 should show a result of opta/commands/echo 20. Attach a screenshot of the third and fourth command windows to show that the expected responses were received.	See below
7	Send command of 3	USend the following command to the second command window to publish the input to the Opta: mosquitto_pub -h 172.16.0.1 -t "opta/commands/input" -m "3". Window 3 should show a result of opta/commands/echo 3. Window 4 should show a result of opta/commands/echo 6. Attach a screenshot of the third and fourth command windows to show that the expected responses were received.	See below
8	Tear Down	Disconnect the Opta by unplugging it. Within 10 seconds, Mosquitto will publish the Last Will message. Confirm the message has been received by entering the command mosquitto_sub -h 172.16.0.1 -t	See below

		“opta/status” -v --retained-only. The expected result is opta/status offline. Attach a screenshot of the second command window to show that the expected response was received. Stop the mosquitto broker by going to the first command window and pressing Ctrl+C.	
--	--	--	--

Step 2: Verify Connection

```
C:\mosquitto>mosquitto_sub -h 10.0.0.100 -t "opta/status" -v
opta/status online
```

Step 3: Read Telemetry

```
C:\mosquitto>mosquitto_sub -h 10.0.0.100 -t "opta/device-id" -v --retained-only
opta/device-id 1234
```

Step 4 and Step 5: Read Heartbeat

```
C:\mosquitto>mosquitto_sub -h 10.0.0.100 -t "opta/heartbeat" -v -C 1
opta/heartbeat 8

C:\mosquitto>mosquitto_sub -h 10.0.0.100 -t "opta/heartbeat" -v -C 1
opta/heartbeat 5
```

Step 6: Send command of 10

```
C:\Users\Sindrani>mosquitto_sub -h 10.0.0.100 -t "opta/commands/echo" -v -C 1
opta/commands/echo 10
```

```
C:\Users\Sindrani>mosquitto_sub -h 10.0.0.100 -t "opta/commands/result" -v -C 1
opta/commands/result 20
```

Step 7: Send command of 3

```
C:\Users\Sindrani>mosquitto_sub -h 10.0.0.100 -t "opta/commands/echo" -v -C 1
opta/commands/echo 3
```



```
C:\Users\Sindrani>mosquitto_sub -h 10.0.0.100 -t "opta/commands/result" -v -C 1  
opta/commands/result 6
```

Step 8: Tear Down

```
C:\mosquitto>mosquitto_sub -h 10.0.0.100 -t "opta/status" -v --retained-only  
opta/status offline
```

Arduino OpenADR Test

Purpose

This test establishes a framework for OpenADR communications on the Arduino Opto which can be used for future integration. The test will register the Arduino as a shedding device, set up a shedding command from the laptop, and then have the Arduino send a report about the response to the shedding command.

Description

In this test, the testing laptop is set up as the Virtual Top Node (VTN) using the python openleadr library, which implements OpenADR using open source code. The laptop will run at IP address 10.0.0.100 and the Arduino will run at IP address 10.0.0.101. The Arduino will be set up to poll the VTN on the test laptop once per second.

An XML template file must be uploaded to the Arduino for the test to work. This will be accomplished by starting a WiFi server on the Arduino and then uploading the file from the laptop to the Arduino using the POST method of the curl command.

The first step of the test is to start the VEN running on the Arduino. This is done by starting the opta_ven.ino file. It will publish a message on the Arduino Serial Monitor indicating that the WiFi has been connected and the XML template has been parsed.

The second step of the test is to start the VTN running on the laptop. This is done by starting the vtn_test_server.py python script. This will publish an output in the command line indicating that the VTN has started and showing the Host and Opta IP addresses. Within a few seconds, the Opta will register automatically. This Arduino Serial monitor will show that the VEN has registered and the command prompt will indicate that it recognizes the VTN. The command prompt will give a message telling the user to send the light shed command.

The third step of the test will be to send a light Shed command. This is accomplished by pressing Enter in the command prompt. The command prompt will register that the command prompt was sent, then the Opta will show it was received by printing a response acknowledgement in the Serial monitor window. The command prompt will then output a message saying that the compliance report was received from the VEN and then giving a message to the user to send the moderate shed command.

The fourth step of the test will be to send a moderate Shed command. This is accomplished by pressing Enter in the command prompt. The command prompt will register that the command prompt was sent, then the Opta will show it was received by printing a response acknowledgement in the Serial monitor window. The command prompt will then output a message saying that the compliance report was received from the VEN.

The final step is to tear down the test. First the VTN server is stopped by pressing Ctrl+C in the command prompt. This will output a message saying the VTN has been stopped. In a few seconds the Arduino will indicate that it has lost the connection by publishing a reconnect failed message. After this, unplug the Opta from power to turn it off.

Procedure

	Step	Description	Result
0	Initial Conditions	The Arduino and the laptop are connected to the same subnet using wifi. The laptop's IP address is 10.0.0.100. Open a command line interface and use the command ipconfig. Take a screen shot of the Wireless Lan adapter Wi-fi section and confirm that the IPv4 address is 10.0.0.100.	See below
1	Flash the upload utility to the Arduino Opta.	Flash the upload_template.ino file to the Arduino. A message will be printed in the Arduino IDE's Serial Monitor showing that the WiFi has been connected at IP address 10.0.0.101. Take a screenshot of this message and post it in the results column.	See below
2	Upload the XML template	Navigate to the folder containing openadr_template.xml in the command line prompt and run the command curl -X POST http://10.0.0.101:8888 -H "X-Filename: openadr_template.xml" -data-binary @openadr_template.xml . The message "Upload OK" should appear in the command line. Take a screenshot of this message and post it in the results column.	See below
3	Flash the VEN sketch to the Arduino Opta.	Flash the opta_ven.ino file to the Arduino. A message will be printed in the Arduino IDE's Serial Monitor showing that the template has been parsed and the WiFi is connected with the IP address 10.0.0.101. Take a screenshot of this message and post it in the results column.	See Below
4	Start the VTN server on the laptop and verify that the VEN enrolls	Navigate to the folder containing the vtn_test_server.py script in the command line prompt. Run this script with the command py vtn_test_server.py .	See Below

	properly.	The command line prompt will display a message indicating that the OpenADR VTN has started and that it has a Host IP of 10.0.0.100. Take a screenshot of this message in the Results column. After a few seconds the Opta will enroll automatically. The Serial Monitor will show a message indicating that the VEN has been registered and the Device ID is 1234. The command line prompt will indicate that a registration event from VEN 1234 has been received. Take a screenshot of these two messages in the Results column.	
5	Send moderate shed command	Press Enter at the command line interface to trigger the moderate shed command from the prompt. The Serial Monitor will show a message indicating that the SIMPLE signal level 1 command has been received and that a report was sent with shed = 3 kW. Take a screenshot of this message and put it in the Results column. The command line interface will show the compliance report has been received from the VEN once the response has been sent. It will indicate that the load shed value was 3 kW. Take a screenshot of this message and put it in the Results column.	Openleadr reporting template is not available for the version we're using. The opta indicates the shed command was received and it attempted to send the report (see below). Future work will ensure this reporting works with customer equipment.
6	Send high shed command	Press Enter at the command line interface to trigger the high shed command from the prompt. The Serial Monitor will show a message indicating that the SIMPLE signal level 2 command has been received and that a report was sent with shed = 10 kW. Take a screenshot of this message and put it in the Results column. The command line interface will show the compliance report has been received from the VEN once the response has been sent. It will indicate that the load shed value was 10 kW. Take a screenshot of this message and put it in the Results column.	Skipped (see step 5)
7	Tear Down	Press Ctrl+C in the command line interface to stop the VTN server. A message saying that the VTN stopped should be shown in	Skipped (see step 5)

		the command line. Take a screenshot of this message and put it in the Results column. After a few seconds the Opta will detect that the connection was lost. It will show a message in the Serial Monitor that the Reconnect failed along with an error code. Take a screenshot of this message and put it in the Results column. Unplug the Arduino Opta.	
--	--	---	--

Step 0: Initial Condition

```
Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix  . : 
IPv6 Address. . . . . : 2601:282:1:b840::5bc5
IPv6 Address. . . . . : 2601:282:1:b840:ffed:41b1:dad8:35e7
Temporary IPv6 Address. . . . . : 2601:282:1:b840:30c4:38d:8526:4d48
Temporary IPv6 Address. . . . . : 2601:282:1:b840:9974:6267:bf55:e5d2
Temporary IPv6 Address. . . . . : 2601:282:1:b840:e19a:f79:2b9e:78a8
Link-local IPv6 Address . . . . . : fe80::32b6:a03a:7a16:d579%11
IPv4 Address. . . . . : 10.0.0.100
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::c650:9cff:fe9d:ef97%11
                          10.0.0.1
```

Step 1: Flash Upload Utility

```
Opta File Upload Utility
=====
Initializing block device... OK
Setting up partition table... OK
Mounting filesystem... not found. Formatting user partition only...
OK
Connecting to WiFi
WiFi connected to: Trumbull
Static IP configured successfully. IP: 10.0.0.101
HTTP server started on port 8888
```

Step 2: Upload file

```
Upload OK: /fs/openadr_template.xml
Written : 3019bytes
Verified: 3019 bytes on flash
```

```

Client connected.
Request received (3204 bytes)
Writing to /fs/openadr_template.xml ... OK (3019 bytes)
Upload OK: /fs/openadr_template.xml
Written : 3019bytes
Verified: 3019 bytes on flash

Files on /fs/:
. (0 bytes)
.. (0 bytes)
openadr_template.xml (3019 bytes)

Client disconnected.

```

Step 3: Flash VEN

```

Loading template from /fs/openadr_template.xml
Template loaded. Size: 3019 bytes
Templates parsed into RAM sections:
  oadrRegisterReport : 311 bytes
  oadrPoll            : 57 bytes
  oadrCreatedEvent    : 321 bytes
  oadrUpdateReport    : 247 bytes
Connecting to WiFi
WiFi connected to: Trumbull
Static IP configured successfully. IP: 10.0.0.101

```

Step 4: Start VTN Server

```

21:24:03 INFO      =====
21:24:03 INFO      OpenADR VTN starting
21:24:03 INFO      VTN ID       : test-vtn
21:24:03 INFO      Host        : 10.0.0.100:8080
21:24:03 INFO      Opta IP     : 10.0.0.101
21:24:03 INFO      =====
===== Running on http://0.0.0.0:8080 =====

```

```

--- END REQUEST ---
postToVTN: request sent, waiting for response...
postToVTN: status code: 200
postToVTN: response length: 618
Report registration response received:
<?xml version="1.0" encoding="utf-8"?>
<oadrPayload xmlns="http://openadr.org/oadr-2.0b/2012/07">
<oadr:oadrSignedObject xmlns:oadr="http://openadr.org/oadr-2.0b/2012/07" oadr:id="oadrSignedObject"><oadr:oadrRegisteredReport ei:schemaVersion="2.0b" xmlns:ei="http://docs.oasis-open.org/na/energyinterop/201110"><ei:eiResponse><ei:respo
</oadrPayload>
VEN fully registered. Device ID: 1234

```

Step 5: Moderate Shedding Command

```
postToVIN: request sent, waiting for response...
postToVIN: status code: 200
postToVIN: response length: 591
RunKnowledge response: <?xml version="1.0" encoding="utf-8"?>
<oadrPayload xmlns="http://openadr.org/oadr-2.0b/2012/07">
<oadr:oadrSignedObject xmlns:oadr="http://openadr.org/oadr-2.0b/2012/07" oadr:id="oadrSignedObject"><oadr:oadrResponse ei:schemaVersion="2.0b" xmlns:ei="http://docs.oasis-open.org/ns/energyinterop/201110"><ei:Response><ei:responseCode>
</oadrPayload>
Signal 1: light shed. Reporting 3 MW.
Handling report: shed = 3 MW
postToVIN: attempt 1 of 3
postToVIN: connecting to 10.0.0.100:8080
```